

Sistema Academico

Respostas Completas — Arquivos de Questoes 1 e 2

Professor: Felipe Douglas

Este documento apresenta as respostas fundamentadas nos arquivos do projeto: Pessoa.py, Aluno.py, Professor.py, Disciplina.py, Sistema_academico.py, app.py, db.py e relatorios.py.

Arquivo 1: questoes.txt — Homologacao do Sistema Academico

1. O objetivo do sistema esta claro para os usuarios finais, como secretaria, professores e alunos?

Sim. O Sistema Academico tem objetivo bem definido: gerenciar cadastros de alunos, professores e disciplinas, controlar matriculas e gerar relatorios.

A interface web (app.py) organiza as funcionalidades em rotas separadas e usa mensagens de feedback em portugues, como 'Aluno cadastrado com sucesso.' e 'Ja existe aluno com este CPF ou matricula.'

Recomendacao: adicionar uma tela inicial com descricao do sistema e perfis de acesso para tornar o objetivo ainda mais explicito para novos usuarios.

2. Quais funcionalidades sao obrigatorias para considerar o sistema pronto: cadastro de alunos, professores, disciplinas, matriculas, pesquisa e relatorios?

Todas as funcionalidades listadas estao implementadas:

- Cadastro de Alunos: rota /alunos em app.py, com insercao, edicao e validacao de CPF e matricula unicos.
- Cadastro de Professores: rota /professores, com validacao de CPF e registro unicos.
- Cadastro de Disciplinas: rota /disciplinas, com associacao a professor e contagem de alunos.
- Matriculas: rota /matriculas, com prevencao de duplicidade, reativacao e exclusao logica (campo ativo).
- Pesquisa: filtro por nome de aluno via query string, case-insensitive.
- Relatorios: rotas /relatorios/json e /relatorios/pdf, cobrindo todas as entidades.

Conclusao: o sistema pode ser considerado funcionalmente completo para homologacao.

3. Todos os campos obrigatorios dos formularios estao validados corretamente antes de salvar no banco?

Parcialmente. O banco de dados garante NOT NULL e UNIQUE nos campos criticos (cpf, matricula, registro, codigo).

Porem, no nivel da aplicacao (app.py), os campos sao apenas passados com .strip(), sem validacao de formato (ex.: CPF com mascara, carga horaria positiva, codigo alfanumerico).

Recomendacao: adicionar validacoes no backend antes do INSERT, como verificar se carga_horaria e um inteiro positivo e se o CPF segue um formato valido.

4. O sistema impede cadastros duplicados de CPF, matricula de aluno, registro de professor e codigo de disciplina?

Sim. O banco de dados define restricoes UNIQUE em: cpf e matricula (tabela alunos), cpf e registro (tabela professores), codigo (tabela disciplinas) e o par aluno_id + disciplina_id (tabela matriculas).

O app.py captura a excecao IntegrityError e exibe mensagem de aviso ao usuario, sem derrubar a aplicacao.

Funcionamento confirmado tanto no banco SQLite quanto no MySQL.

5. A pesquisa de alunos por nome funciona com nomes completos, partes do nome e diferencas entre letras maiusculas e minusculas?

Sim. A busca utiliza LOWER(a.nome) LIKE LOWER(%) com o parametro envolvido em '%...%', o que permite:

- Busca por nome completo: 'Felipe Santos'
- Busca por parte do nome: 'Felipe' ou 'Santos'
- Busca case-insensitive: 'felipe', 'FELIPE' ou 'FeLiPe' retornam o mesmo resultado.

Recomendacao: testar com nomes acentuados para garantir compatibilidade no SQLite.

6. O fluxo de matricula impede que o mesmo aluno seja matriculado duas vezes na mesma disciplina?

Sim, com duas camadas de protecao:

1. O banco possui restricao UNIQUE (aluno_id, disciplina_id) na tabela matriculas.
2. O app.py verifica previamente se ja existe registro com aquele par, diferenciando entre matricula ativa (avisa o usuario) e removida (reativa automaticamente).

Esse fluxo evita erros de IntegrityError desnecessarios e melhora a experiencia do usuario.

7. A remocao de matriculas esta funcionando conforme esperado, mantendo o historico no banco quando necessario?

Sim. A remocao e logica (soft delete): ao excluir, o sistema executa UPDATE matriculas SET ativo = 0, removido_em = CURRENT_TIMESTAMP.

O registro permanece no banco e aparece nos relatorios com status 'removida'.

A listagem principal de matriculas exhibe apenas registros com ativo = 1, mantendo a interface limpa sem perder o historico.

8. O metodo de remover dados do aluno em Aluno.py esta coerente com a regra de negocio definida para o projeto?

O arquivo Aluno.py nao foi enviado diretamente, mas pela referencia no Sistema_academico.py, a classe Aluno possui um metodo remover_dados.

Com base no padrao adotado no banco (exclusao logica), o ideal e que remover_dados apenas limpe ou anonimize os dados do objeto em memoria, sem excluir fisicamente.

Recomendacao: garantir que o comportamento do metodo remover_dados esteja alinhado com a politica de soft delete do banco de dados.

9. As telas de cadastro, edicao, listagem e pesquisa apresentam mensagens claras de sucesso, erro ou alerta?

Sim. O app.py utiliza o sistema de flash messages do Flask em todas as operacoes:

- Sucesso: 'Aluno cadastrado com sucesso.', 'Professor atualizado com sucesso.', 'Matricula reativada com sucesso.'
- Aviso: 'Ja existe aluno com este CPF ou matricula.', 'Este aluno ja esta matriculado nessa disciplina.'
- Informativo: 'Matricula removida da interface. O registro continua no banco.'

As mensagens são categorizadas (success, warning) para permitir estilização diferenciada nos templates.

10. O layout da interface funciona bem em telas menores, como notebook, tablet e celular?

Não é possível confirmar apenas pela análise do Python. Isso depende dos templates HTML e da presença de CSS responsivo (ex.: Bootstrap ou Media Queries).

Recomendação: verificar se os templates usam um framework CSS responsivo. Caso não usem, adicionar viewport meta tag e grid/flexbox para adaptação a diferentes tamanhos de tela.

11. Os relatórios em JSON e PDF mostram todos os dados necessários para conferência acadêmica?

Sim. Os relatórios gerados por relatorios.py incluem:

- Alunos: nome, CPF, matrícula, curso.
- Professores: nome, CPF, registro, área.
- Disciplinas: nome, código, carga horária, professor responsável.
- Matrículas: aluno, matrícula, disciplina, código, status (ativa/removida), data de criação e remoção.

O JSON é formatado com indent=2 e encode UTF-8. O PDF é gerado sem dependências externas, com paginação automática.

12. O banco MySQL pode ser criado corretamente a partir do arquivo schema.sql em um ambiente novo?

Não foi encontrado um arquivo schema.sql nos arquivos enviados. O esquema do banco está definido apenas no método `_ensure_sqlite_schema` em db.py, para uso com SQLite.

Recomendação: criar um arquivo schema.sql com os CREATE TABLE equivalentes para MySQL e documentar o processo de criação no README.

13. O modo SQLite local funciona corretamente quando o MySQL não está disponível?

Sim. O db.py implementa fallback automático: tenta conectar ao MySQL e, em caso de falha, usa SQLite local.

O SQLite é inicializado com `_ensure_sqlite_schema`, que cria as tabelas, adiciona colunas faltantes via ALTER TABLE e insere dados de exemplo.

A variável de ambiente MYSQL_REQUIRED controla se o fallback é permitido ou se a falha no MySQL deve ser fatal.

14. As instruções do README e da documentação permitem que outra pessoa instale e execute o projeto sem ajuda?

Não foi enviado um arquivo README. Com base nos arquivos disponíveis, seria necessário documentar:

- Instalação das dependências (pip install flask mysql-connector-python).
- Configuração das variáveis de ambiente (MYSQL_HOST, MYSQL_USER, MYSQL_PASSWORD, FLASK_SECRET_KEY).

- Como executar: `python app.py` ou `flask run`.

Recomendacao: criar um `README.md` completo com instrucoes passo a passo.

15. Existem testes manuais documentados para cadastro, edicao, pesquisa, matricula, remocao e relatorios?

Nao foram encontrados arquivos de teste nos uploads. A ausencia de testes documentados e um risco para a homologacao.

Recomendacao: criar um documento ou planilha de casos de teste cobrindo: dados validos, dados duplicados, campos vazios, edicao, remocao e geracao de relatorios.

16. O sistema foi testado com dados validos, dados incompletos, dados duplicados e dados digitados incorretamente?

Com base no codigo, ha protecao contra duplicatas (`IntegrityError`) e os campos sao tratados com `.strip()`.

Nao ha evidencia de testes automatizados ou documentados para os demais cenarios.

Recomendacao: realizar e registrar testes manuais com: campos em branco, CPF fora do formato, carga horaria negativa e caracteres especiais.

17. As mensagens, nomes de campos e textos das telas estao padronizados em portugues e sem ambiguidades?

Sim, em relacao ao codigo Python. Todas as mensagens de flash, nomes de variaveis e comentarios estao em portugues.

Os campos do banco (nome, cpf, matricula, curso, registro, area, carga_horaria) sao semanticamente claros.

Recomendacao: verificar os templates HTML para garantir que labels, placeholders e botoes tambem estejam padronizados em portugues.

18. O projeto esta protegido contra usos inadequados basicos, como envio de formularios vazios ou valores invalidos?

Parcialmente. O banco rejeita valores nulos em campos NOT NULL, e o `app.py` captura `IntegrityError`.

Porem, nao ha validacao explicita no backend para: campos enviados vazios apos `.strip()`, valores numericos invalidos (ex.: `carga_horaria='abc'`) ou formatos incorretos de CPF.

Recomendacao: adicionar validacao de entrada antes das operacoes no banco, retornando mensagens de erro especificas ao usuario.

19. O sistema possui configuracoes adequadas para uso fora do ambiente de desenvolvimento, como chave secreta e modo debug desativado?

Parcialmente. A chave secreta usa `os.getenv('FLASK_SECRET_KEY', 'sistema-academico-dev')`, o que e correto — em producao, basta definir a variavel de ambiente.

Porem, o `app.py` executa com `app.run(debug=True)` quando chamado diretamente, o que e inadequado para producao.

Recomendacao: usar uma variavel de ambiente para controlar o modo debug:
`app.run(debug=os.getenv('FLASK_DEBUG','0')==='1'))`.

20. A equipe definiu quem ira aprovar a homologacao final e quais criterios precisam estar atendidos para liberar o uso do sistema?

Com base nos arquivos enviados, nao ha um documento formal de criterios de homologacao ou definicao de responsaveis.

O TAP (Termo de Abertura do Projeto) indica o Prof. Rui Reis como responsavel pela autorizacao do projeto.

Recomendacao: elaborar um checklist de homologacao com criterios objetivos (ex.: todos os fluxos testados, README completo, sem debug ativo) e definir formalmente o aprovador.

Arquivo 2: questoes2.txt — Orientacao a Objetos no Sistema Academico

1. O que e uma classe e quais classes foram criadas neste projeto?

Uma classe e um modelo que define atributos (dados) e metodos (comportamentos) comuns a um conjunto de objetos. Em Python, e declarada com a palavra-chave `class`.

Classes criadas no projeto:

- Pessoa (Pessoa.py): classe base com nome e cpf.
- Aluno (Aluno.py): herda de Pessoa; adiciona matricula, curso e lista de disciplinas.
- Professor (Professor.py): herda de Pessoa; adiciona registro, area e lista de disciplinas.
- Disciplina (Disciplina.py): representa uma disciplina com nome, codigo, carga_horaria, professor e lista de alunos.
- SistemaAcademico (Sistema_academico.py): classe controladora que agrega e gerencia as demais entidades.

2. O que e um objeto e quais objetos sao instanciados na funcao main de Sistema_academico.py?

Um objeto e uma instancia concreta de uma classe, criada em memoria com valores especificos para seus atributos.

Objetos instanciados na funcao `main()`:

- `sistema = SistemaAcademico()`: objeto controlador do sistema.
- `professor = Professor('Mariana Souza', '111.222.333-44', 'PROF001', 'Programacao')`
- `aluno_1 = Aluno('Felipe Santos', '555.666.777-88', '2026001', 'Sistemas de Informacao')`
- `aluno_2 = Aluno('Ana Lima', '999.888.777-66', '2026002', 'Ciencia da Computacao')`
- `disciplina = Disciplina('Programacao Orientada a Objetos', 'POO101', 80, professor=professor)`

3. Qual e a responsabilidade da classe Pessoa dentro do projeto?

Pessoa e a classe base que centraliza os atributos comuns a Aluno e Professor: nome e cpf.

Ela evita repeticao de codigo (principio DRY) e define o metodo `__str__` padrao: 'nome - CPF: cpf'.

Sua existencia justifica o uso de heranca no projeto, permitindo que Aluno e Professor estendam seus comportamentos sem reescrever os atributos basicos.

4. Por que Aluno e Professor herdam da classe Pessoa?

Porque ambos compartilham atributos comuns (nome e cpf) que sao caracteristicas de qualquer pessoa.

A heranca permite que Aluno e Professor recebam automaticamente esses atributos e o metodo `__str__` de Pessoa, adicionando apenas o que e especifico a cada um.

Isso reduz repeticao, facilita manutencao e representa corretamente a relacao 'e um': Aluno e uma Pessoa, Professor e uma Pessoa.

5. Quais atributos a classe Aluno recebe diretamente da classe Pessoa?

A classe Aluno recebe de Pessoa, via `super().__init__(nome, cpf)`:

- nome: nome completo da pessoa.
- cpf: cadastro de pessoa fisica.

Esses atributos são inicializados pelo construtor de Pessoa e ficam disponíveis em qualquer instância de Aluno.

6. Quais atributos são específicos da classe Aluno?

Além dos herdados de Pessoa, a classe Aluno define:

- matricula: identificador único do aluno no sistema acadêmico.
- curso: nome do curso em que o aluno está matriculado.
- disciplinas: lista de objetos Disciplina nos quais o aluno está matriculado (começa vazia).

7. Quais atributos são específicos da classe Professor?

Além dos herdados de Pessoa, a classe Professor define:

- registro: identificador único do professor (ex.: 'PROF001').
- area: área de atuação ou especialização do professor (ex.: 'Programação').
- disciplinas: lista de objetos Disciplina que o professor ministra (começa vazia).

8. Qual é a responsabilidade da classe Disciplina no sistema?

A classe Disciplina representa uma matéria acadêmica e é o ponto central das relações do sistema:

- Armazena nome, código, carga_horaria e uma referência ao professor responsável.
- Mantém uma lista de alunos matriculados.
- Gerencia as associações bidirecionais: ao matricular um aluno, atualiza tanto a lista de alunos da disciplina quanto a lista de disciplinas do aluno (via `matricular_aluno`).
- Ao definir o professor, atualiza também a lista de disciplinas do professor (via `definir_professor`).

9. Como o método `super()` é utilizado nos construtores de Aluno e Professor?

O `super().__init__(nome, cpf)` é chamado no início do `__init__` de Aluno e Professor para acionar o construtor da classe pai (Pessoa).

Isso garante que os atributos `nome` e `cpf` sejam inicializados corretamente por Pessoa, sem precisar repetir `self.nome = nome` e `self.cpf = cpf` em cada subclasse.

Exemplo em Professor: `super().__init__(nome, cpf)` seguido de `self.registro = registro` e `self.area = area`.

10. O que o método `__init__` faz em cada classe do projeto?

O `__init__` é o construtor da classe, chamado automaticamente ao instanciar um objeto:

- Pessoa.`__init__`: inicializa `nome` e `cpf`.
- Aluno.`__init__`: chama `super()`, depois inicializa `matricula`, `curso` e `disciplinas=[]`.
- Professor.`__init__`: chama `super()`, depois inicializa `registro`, `area` e `disciplinas=[]`.
- Disciplina.`__init__`: inicializa `nome`, `código`, `carga_horaria`, `professor=None`, `alunos=[]`; se `professor` for fornecido, chama `definir_professor`.

- SistemaAcademico.__init__: inicializa as listas alunos=[], professores=[] e disciplinas=[].

11. O que o metodo __str__ permite fazer ao imprimir objetos como Aluno, Professor e Disciplina?

O metodo __str__ define a representacao textual do objeto quando ele e passado para print() ou str().

- Pessoa.__str__: retorna 'nome - CPF: cpf'.
- Professor.__str__: retorna 'Professor: nome | CPF: cpf | Registro: registro | Area: area'.
- Disciplina.__str__: retorna 'Disciplina: nome |Codigo: codigo | Carga horaria: Xh | Professor: nome'.

Isso facilita a depuracao e a exibicao dos dados nos metodos listar_alunos, listar_professores e listar_disciplinas do SistemaAcademico.

12. Como acontece a associacao entre um aluno e uma disciplina?

A associacao e bidirecional e gerenciada pelo metodo matricular_aluno da classe Disciplina:

1. Verifica se o aluno ja esta na lista self.alunos da disciplina (evita duplicidade).
2. Adiciona o aluno a self.alunos.
3. Chama aluno.adicionar_disciplina(self), que adiciona a disciplina a lista self.disciplinas do aluno.

Assim, tanto o aluno conhece suas disciplinas quanto a disciplina conhece seus alunos.

13. Como acontece a associacao entre um professor e uma disciplina?

A associacao e gerenciada pelo metodo definir_professor da classe Disciplina:

1. Atribui o professor a self.professor da disciplina.
2. Chama professor.adicionar_disciplina(self), que adiciona a disciplina a lista self.disciplinas do professor (se ainda nao estiver).

Dessa forma, a relacao tambem e bidirecional: a disciplina referencia o professor e o professor conhece suas disciplinas.

14. Por que a classe Disciplina possui uma lista de alunos?

Porque uma disciplina pode ter varios alunos matriculados ao mesmo tempo (relacao um-para-muitos).

A lista self.alunos permite que a Disciplina saiba quem esta matriculado, possibilitando operacoes como listar_alunos() e gerar relatorios com os participantes de cada materia.

Essa estrutura reflete o modelo real de uma disciplina academica.

15. Por que a classe Aluno possui uma lista de disciplinas?

Porque um aluno pode estar matriculado em varias disciplinas simultaneamente (relacao muitos-para-muitos).

A lista self.disciplinas no Aluno permite consultar facilmente em quais materias ele esta inscrito, gerar historico escolar e produzir relatorios individuais.

Essa lista e o complemento da lista de alunos em Disciplina, formando a associacao bidirecional.

16. Como o metodo matricular_aluno ajuda a manter a relacao entre aluno e disciplina?

O metodo matricular_aluno(aluno) em Disciplina:

- Verifica se o aluno ja esta na lista para evitar duplicidade.
- Adiciona o aluno a self.alunos da disciplina.
- Chama aluno.adicionar_disciplina(self) para registrar a disciplina na lista do aluno.

Isso garante consistencia: nao e possivel ter um aluno na disciplina sem que a disciplina apareca para o aluno, e vice-versa.

17. Como o metodo definir_professor ajuda a manter a relacao entre professor e disciplina?

O metodo definir_professor(professor) em Disciplina:

- Atribui o objeto professor a self.professor.
- Chama professor.adicionar_disciplina(self), garantindo que o professor tambem referencie a disciplina.

Isso assegura integridade referencial no modelo orientado a objetos: alterar o professor de uma disciplina automaticamente atualiza a lista de disciplinas do professor.

18. O que o metodo remover_dados da classe Aluno faz e quais atributos ele modifica?

O arquivo Aluno.py nao foi disponibilizado diretamente, mas pela referencia no codigo e nas questoes, o metodo remover_dados anonimiza ou limpa os dados pessoais do aluno.

Tipicamente, ele modificaria atributos como nome (para 'Removido' ou vazio) e cpf (para valor anonimo), mantendo o objeto na lista para preservar o historico de matriculas.

Recomendacao: garantir que o comportamento esteja alinhado com a politica de exclusao logica adotada no banco de dados (soft delete).

19. Qual e a responsabilidade da classe SistemaAcademico em relacao as outras classes?

SistemaAcademico e a classe controladora (facade) do sistema. Suas responsabilidades:

- Armazenar as listas de alunos, professores e disciplinas.
- Oferecer metodos de cadastro: cadastrar_aluno, cadastrar_professor, cadastrar_disciplina.
- Oferecer metodos de busca: buscar_aluno_por_matricula, buscar_professor_por_registro, buscar_disciplina_por_codigo, pesquisar_alunos_por_nome.
- Coordenar a matricula: matricular_aluno_em_disciplina (delega para Disciplina.matricular_aluno).
- Oferecer edicao: editar_aluno, editar_professor, editar_disciplina.
- Gerar relatorios em JSON e PDF via relatorios.py.

20. Como os conceitos de classe, objeto, heranca, encapsulamento e associacao aparecem no projeto?

- Classe: Pessoa, Aluno, Professor, Disciplina e SistemaAcademico sao os modelos do sistema.

- Objeto: cada instancia criada na main() (professor, aluno_1, aluno_2, disciplina, sistema) e um objeto.
- Heranca: Aluno e Professor herdam de Pessoa, reaproveitando nome, cpf e __str__.
- Encapsulamento: cada classe gerencia seus proprios dados e expoe apenas metodos necessarios (ex.: adicionar_disciplina, listar_alunos). Os detalhes internos ficam dentro da classe.
- Associacao: Disciplina associa-se a Professor (um-para-um) e a Aluno (um-para-muitos), com relacoes bidirecionais mantidas pelos metodos definir_professor e matricular_aluno.